

# PM\_Assignment\_Ecommerce\_MVP\_Solution

- steve hu

## Part 1: Product Design

### 1. User Roles and Core Scenarios

| Role                        | Core Scenarios                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customer(Buyer)             | <ul style="list-style-type: none"><li>- Browse products via categories, search, or promotions.</li><li>- View product details (images, price, specs, reviews).</li><li>- Add/remove items from the shopping cart.</li><li>- Adjust product quantities in the cart.</li><li>- Submit an order and complete payment.</li><li>- View order history and track shipment status.</li><li>- Request order cancellation or refund.</li></ul> |
| Operations Staff (Internal) | <ul style="list-style-type: none"><li>- Create, edit, and publish new products.</li><li>- Manage product inventory and SKU levels.</li><li>- Review and process incoming orders.</li><li>- Update order status (e.g., confirm, ship).</li><li>- Handle customer cancellation/refund requests.</li><li>- View basic sales and inventory reports.</li></ul>                                                                            |
| Platform Admin              | <ul style="list-style-type: none"><li>- Manage user permissions for operations staff.</li><li>- View platform-level metrics (GMV, order volume, etc.).</li><li>- Handle system-wide configurations (e.g., payment timeout).</li><li>- Resolve abnormal orders (e.g., payment issues, system errors).</li><li>- Oversee operations staff actions (audit log).</li></ul>                                                               |

### 2. MVP Feature list and priority

| Module                 | Feature                                          | Priority | Reason                                                                        |
|------------------------|--------------------------------------------------|----------|-------------------------------------------------------------------------------|
| User(customer side)    | Login / Registration                             | P0       | Needed to associate orders and payments with a user identity.                 |
| Product(customer side) | Product List (basic, no search/filter initially) | P0       | The entry point for customers to see what's for sale.                         |
| Product(customer side) | Product detail page                              | P0       | Essential for customers to see price, description, and add to cart.           |
| Cart(customer side)    | Add to Cart / View Cart                          | P0       | Standard shopping pattern. Allows customers to collect items before purchase. |
| Cart(customer side)    | Update item quantity in                          | P0       | Basic card functionality.                                                     |

|                          |                                           |    |                                                                                   |
|--------------------------|-------------------------------------------|----|-----------------------------------------------------------------------------------|
|                          | Cart                                      |    |                                                                                   |
| Order(customer side)     | Submit order                              | P0 | The action that creates a formal order and locks inventory.                       |
| Payment(customer side)   | Make Payment (mock/simulated gateway)     | P0 | Completes the core transaction. A real gateway can be P1, but a mock is P0.       |
| Order(customer side)     | Order List / Order Detail                 | P0 | Customers need to see their order history and current status.                     |
| Order(customer side)     | Cancel Order (within a time limit)        | P1 | Improves user experience and reduces support tickets for mistaken orders.         |
| Product(customer side)   | Product Search / Category Filter          | P1 | Significantly improves product discovery for a better UX.                         |
| Payment(customer side)   | Refund Request                            | P1 | Necessary for customer satisfaction, but can be manually processed initially.     |
| Product(admin side)      | Product Management (CRUD)                 | P0 | There is nothing to sell without products.                                        |
| Inventory(admin side)    | SKU / Inventory Management                | P0 | Essential to prevent overselling and manage stock.                                |
| Order(admin side)        | Order Management (view, update status)    | P0 | Staff must be able to see and process orders (e.g., mark as shipped).             |
| Shipment(admin side)     | Shipment Management (add tracking number) | P0 | Completes the "Ship" step in the core transaction flow.                           |
| User(admin side)         | User Management (basic)                   | P1 | Needed to add/remove operations staff, though a single admin account could be P0. |
| Data(admin side)         | Basic Data Dashboard (e.g., sales volume) | P2 | Nice to have for insights, but not critical for the transaction flow.             |
| Promotion(customer side) | Coupons / Discounts                       | P2 | A nice-to-have for future iterations, not core to MVP transaction.                |
| Payment(customer side)   | Multiple Payment                          | P2 | Start with one (e.g., mock/card) for MVP.                                         |

|  |         |  |  |
|--|---------|--|--|
|  | Methods |  |  |
|--|---------|--|--|

### 3. Core Process Design

Chosen Processes:

1. Customer order and payment process
2. Order shipment process

#### Process 1: Customer order and payment process

##### Normal Flow:

1. Customer adds product(s) to cart.
2. Customer clicks "Submit Order" on the cart/checkout page.
3. System creates an order with status PENDING\_PAYMENT, locks inventory, and starts a payment timer (e.g., 30 minutes).
4. Customer is redirected to Payment Gateway.
5. Customer enters payment details and confirms.
6. Gateway returns "Payment Successful".
7. System updates order status to PAID.
8. System triggers inventory deduction (finalizes the lock) and sends a notification to the operations team.
9. Customer sees "Payment Successful" page and order status PAID.

##### Exception Scenarios:

| Exception                | System handling                                                                                                                                                                                                                                                              |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Insufficient Inventory   | At step 2 (Submit Order), system checks inventory. If insufficient, prevents order creation and shows error: "Sorry, some items in your cart are out of stock. Please remove them and try again." The cart item is marked as invalid.                                        |
| Payment Failure          | At step 6, the gateway returns "Payment Failed". System keeps order status as PENDING_PAYMENT. The page shows the failure reason (e.g., "Insufficient funds"). Customer can retry payment using the same order (re-attempt step 4).                                          |
| Payment Timeout          | The payment timer expires (e.g., 30 minutes). System automatically cancels the order, changes status to CANCELLED, and releases the locked inventory. Customer sees the order as "Cancelled - Payment timeout" in their order list.                                          |
| Product becomes OFF_SALE | If a product in the user's cart is taken off sale before order submission: At step 2, the system re-validates product status. If OFF_SALE, it prevents order creation and shows error: "Product [Name] is no longer available. Please remove it from your cart to continue." |

**Status Changes:**

DRAFT (in cart) → PENDING\_PAYMENT (order submitted) → PAID (payment success)  
→ ...to Shipment Process.

PENDING\_PAYMENT → CANCELLED (on timeout or user cancel).

**Process 2: Order Shipment Process****Normal Flow:**

1. Operations staff logs into the admin panel and navigates to "Orders" list.
2. Staff filters by status PAID.
3. Staff selects an order and clicks "Mark as Ready to Ship".
4. Staff packs the goods and prints the packing slip.
5. Staff arranges pickup with a logistics partner and receives a tracking number.
6. Staff enters the tracking number and carrier name into the system, then clicks "Mark as Shipped".
7. System updates order status from PAID to SHIPPED.
8. An automated notification (email/SMS) is sent to the customer with the tracking information.
9. Customer can now see the SHIPPED status and tracking number on their Order Detail page.
10. After a reasonable delivery time (e.g., 7 days) or upon customer confirmation, the system or staff marks the order as COMPLETED.

**Exception Scenarios:**

| Exception                            | System handling                                                                                                                                                                                                                                                                  |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Item is out of stock after payment   | This indicates a serious system flaw (overselling). The system should not allow this. If it happens (e.g., due to race condition), the process halts at PAID. Admin must handle manually: contact customer, offer refund or substitute, and mark order as REFUNDED or CANCELLED. |
| Logistics tracking number is invalid | Staff enters a number that is clearly wrong (e.g., format check fails). System should provide a warning before saving, but allow override. The order status remains PAID until a valid number is provided.                                                                       |
| Order is lost in transit             | Customer contacts support. Admin can mark the order as COMPLETED with a note, or initiate a refund process. Status could go to REFUNDING if a refund is agreed.                                                                                                                  |

**Status Changes:**

PAID → SHIPPED (tracking number added) → COMPLETED (delivery confirmed).

PAID → REFUNDING / REFUNDED (handled by separate cancellation/refund process if item not shipped).

**4. Simplified PRD**

Chosen Feature: Shopping Cart.

### Feature Background:

The shopping cart is a core component of the e-commerce transaction flow. It allows customers to collect multiple items they intend to purchase, review their selection, modify quantities, and proceed to a single, unified checkout. Without a cart, customers would have to purchase each item individually, leading to a poor user experience.

### User Goal:

The user is able to:

Add one or more products from the product detail or list page.

See a summary of selected items, individual prices, and total order value.

Increase or decrease the quantity of a specific item.

Remove an item entirely from the cart.

Easily proceed to the "Submit Order" step.

### Page fields:

| Field Name                        | Data Source / Logic            | Notes                                                 |
|-----------------------------------|--------------------------------|-------------------------------------------------------|
| Product Image                     | product.images[0]              | Thumbnail link.                                       |
| Product Name                      | product.productName            | Link to product detail page.                          |
| SKU Attribute (e.g., Color, Size) | cartItem.selectedSkuAttributes | Display if applicable.                                |
| Unit Price                        | sku.price                      | Current selling price.                                |
| Quantity                          | cartItem.quantity              | Input field (number picker).<br>Min=1, Max=sku.stock. |
| Item Subtotal                     | Unit Price * Quantity          | Calculated client-side or server-side.                |
| Remove Button                     | (Action)                       | Removes the item from the cart.                       |
| Total Order Amount                | Sum of all Item Subtotal       | Displayed prominently.                                |
| Proceed to Checkout button        | (Action)                       | Submits the cart for order creation.                  |

### Business Rules:

1. Inventory Validation: When a user loads the cart page, the system must re-fetch the latest price and stock for each sku in the cart. Items with stock  $\leq 0$  or status  $\neq$  'ON\_SALE' should be marked as "invalid" and moved to a separate section or grayed out.
2. Price Changes: If a product's price has increased since being added to the cart, the user should be notified (e.g., "Price for [Product] has changed from \$19.99 to \$21.99"). The new price is applied.
3. Quantity Limits: The user cannot set quantity to more than the available stock. The input field max should be dynamically bound to sku.stock.

4. Cross-Session Persistence: For logged-in users, the cart must be saved to the database and persist across browser sessions. For guests, the cart can be stored in local storage until login.

**Error Messages / Exception Handling:**

| Exception                                                           | User-facing message                                                                               |
|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Trying to add more than current stock to cart.                      | "Sorry, only [X] items are available."                                                            |
| An item in the cart is now out of stock when loading the cart page. | "[Product Name] is currently out of stock and has been moved to the 'Unavailable Items' section." |
| An item in the cart has been taken off sale.                        | "[Product Name] is no longer available for purchase. Please remove it from your cart."            |
| Trying to proceed to checkout with an empty cart.                   | "Your cart is empty. Please add some products before checking out."                               |

**Acceptance Criteria:**

1. As a user, I can add a product from its detail page by clicking "Add to Cart". A success message appears.
2. The cart page displays all added items with correct images, names, prices, and quantities.
3. I can increase or decrease the quantity of an item using +/- buttons. The item subtotal and total order amount update instantly.
4. I can remove an item by clicking a "Remove" button. The item disappears, and totals are recalculated.
5. If I try to set a quantity to 0 or negative, the item is removed from the cart.
6. If the quantity exceeds available stock, the system prevents the change and shows an error message.
7. If an item in my cart is no longer available (OFF\_SALE or stock=0), it is clearly marked as "unavailable" and I cannot check out with it.
8. When I click "Proceed to Checkout", the system creates a new order with the cart's current validated items and navigates to the order submission page.

## Part 2: Code/Technical Understanding

**5. API Understanding Answers:**

**Question1:** What information can be displayed on the product detail page based on this API response?

**Answer:**

Product ID (for internal tracking).

Product Name (Bluetooth Earphones).

Current Selling Price (19900 cents).

Original/MRP Price (29900 cents).  
Sales Count (e.g., "132 sold").  
Product Images (list of image URLs).  
Product Status (ON\_SALE or OFF\_SALE).  
Total product inventory (sum of all SKU stocks, which is 25).

**Question2:** How should price: 19900 be displayed to users on the page?

**Answer:**

The value is in cents. The page should divide by 100 to get dollars/yuan. It should be formatted as a currency: ¥199.00. The same for originalPrice: ¥299.00.

**Question3:** If status = OFF\_SALE, how should the page handle this product?

**Answer:**

The "Add to Cart" and "Buy Now" buttons should be disabled or hidden.  
A clear label should be displayed, such as "Product is currently unavailable" or "Out of Stock" (if that status is distinct).  
The price and other details can still be shown for informational purposes.  
The user should not be able to purchase the product.

**Question4:** If the total product stock is 0, how should the purchase button be displayed?

**Answer:**

The purchase button (e.g., “Add to Cart” or “Buy Now” ) should be disabled and its text changed to “Out of Stock” or “Sold Out” .  
It should not be clickable. Additionally, the product detail page may show a grayed-out button or remove the button entirely and replace it with plain text.

**Question5:** If the user selects the Black SKU, how should the displayed price and inventory change?

**Answer:**

When the user selects the Black SKU (skuld: 20002), the frontend should update the displayed price and inventory based on the selected SKU's data:  
Price → Change from 19900 (¥199.00) to 20900 (¥209.00)  
Note: If the original price is also shown, it typically remains at the product-level originalPrice (29900), unless the SKU has its own original price (not provided in this response).  
Inventory → Change from 25 (product-level stock) to 15 (Black SKU stock).

**Question6:** What fields do you think are missing from this API response? Please list 3-5 fields and explain why they are needed.

**Answer:**

| Missing field           | Reason for need                                                        |
|-------------------------|------------------------------------------------------------------------|
| description (HTML/text) | Essential for the product detail page content. Customers need detailed |

|                               |                                                                                                                                        |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|                               | information, specs, and features to make a purchase decision.                                                                          |
| categoryName / categoryId     | Needed for breadcrumb navigation, category filtering, and displaying related products.                                                 |
| shippingWeight / shippingInfo | Shipping cost is a critical part of the purchase decision. Even for MVP, a basic "Free Shipping" or "Weight: 0.5kg" field is valuable. |
| averageRating & reviewCount   | Social proof is a major driver of conversion. Displaying star ratings and the number of reviews builds trust.                          |
| isInUserWishlist (boolean)    | A UX improvement. Allows the product detail page to show a filled heart icon if the user has already favorited this product.           |

## 6. Simple Logic Reading Answers:

**Question1:** What is this function mainly used to determine?

**Answer:**

It determines whether an order can be submitted for a given product SKU, based on:

The product's sale status (productStatus).

The SKU's current stock (skuStock).

The quantity the user wants to buy (buyCount).

**Question2:** Under what conditions can the user not submit the order?

**Answer:**

The user cannot submit an order in three scenarios:

| Condition                   | Returned reason                                 |
|-----------------------------|-------------------------------------------------|
| productStatus !== 'ON_SALE' | 'Product is off sale'                           |
| skuStock <= 0               | 'Insufficient inventory'                        |
| buyCount > skuStock         | 'Purchase quantity exceeds available inventory' |

**Question3:** If productStatus = 'ON\_SALE', skuStock = 3, and buyCount = 5, what result should this function return?

**Answer:**

Because buyCount (5) > skuStock (3), the function returns:

```
{ canSubmit: false, reason: 'Purchase quantity exceeds available inventory' };
```

**Question4:** What additional checks do you think should be added to this logic?

**Answer:**

The current logic is a basic availability + quantity check. For a real e-commerce order submission, the following additional checks are recommended:

1. User login status: Is the user logged in? Anonymous users cannot submit orders.
2. Purchase limit per user: Is there a per-customer purchase limit for this product (e.g., limit 2 per user)? This would require checking past orders.
3. Product active time window: Is the product within its valid sale date range (e.g., not yet launched, or past campaign end date)?
4. User blacklist/status: Is the user's account in good standing (e.g., not banned or flagged for fraud)?
5. Valid address/shipping region: Does the user have a shipping address selected, and can the product be shipped to that region?
6. Cart concurrency: Has the cart contents changed since the user viewed the page (protects against stale data)?

## **7. Order Status Understanding Answers:**

**Question1:** What should be the initial status after a customer submits an order?

**Answer:**

PENDING\_PAYMENT

The order is created but payment has not been completed yet.

**Question2:** What should the status become after payment is successful?

**Answer:**

PAID

Payment is confirmed. The merchant should now prepare for shipment.

**Question3:** What should the status become after the merchant ships the order?

**Answer:**

SHIPPED

The order has left the warehouse. Tracking information can be provided to the customer.

**Question4:** What should the status become if the customer does not pay within the payment time limit?

**Answer:**

CANCELLED

The system automatically cancels the order and releases any reserved inventory.

**Question5:** In which statuses should the customer be allowed to request a refund?

**Answer:**

PAID, SHIPPED, COMPLETED

PAID – Before shipment, the customer can request a full refund (no return needed).

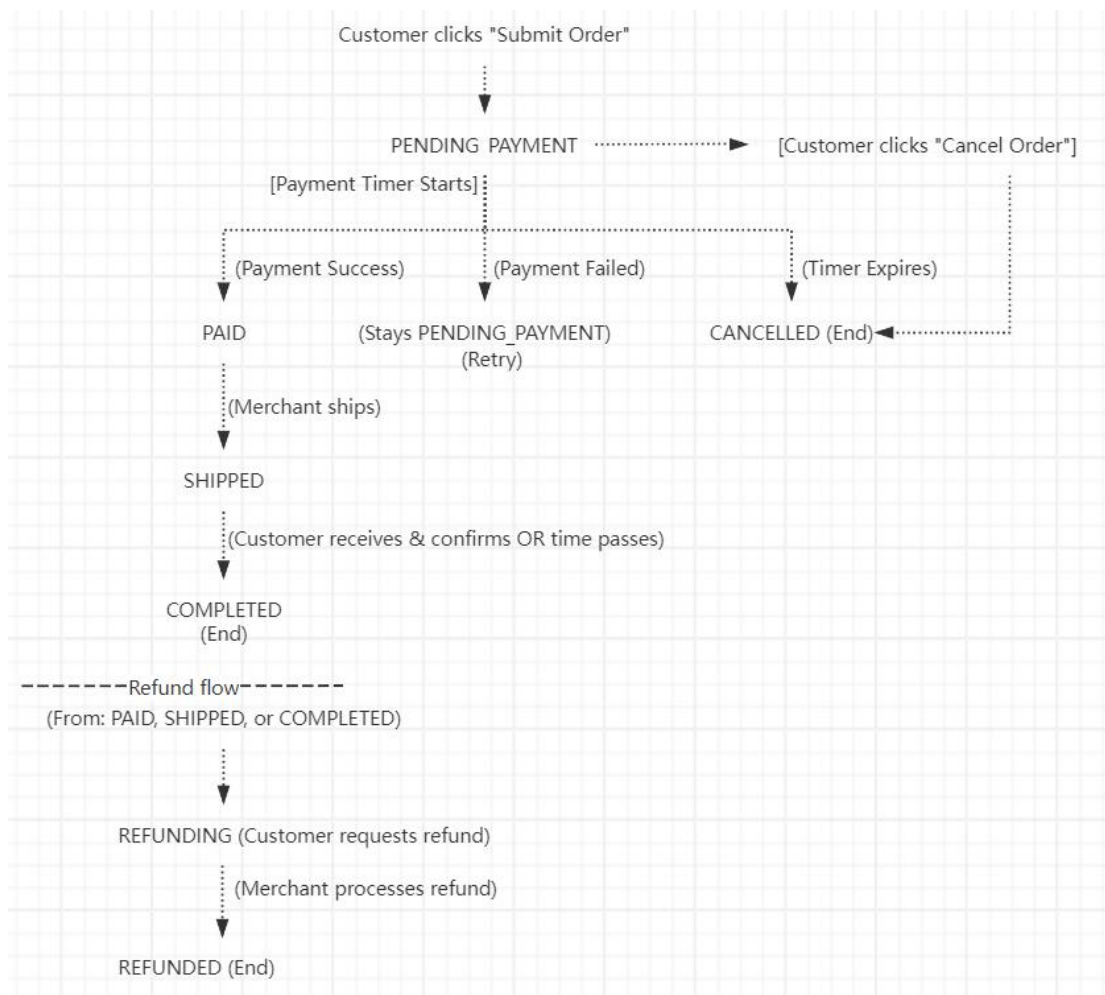
SHIPPED – After shipment but before delivery, refund request may involve canceling shipment or returning.

COMPLETED – Within the return window (e.g., 7–30 days after delivery), customer can request refund/return.

**Question6:** Please draw or describe the order status transition flow.

**Answer:**

(in next page)



## 8. Bug/Exception Analysis Answers:

Analysis as blow:

### Possible Causes:

1. Race Condition / Concurrent Purchase: The most likely cause. Another customer completed their purchase of the last item after User A loaded the product detail page but before User A clicked "Submit Order". The backend correctly validates inventory and rejects the order.
2. Frontend-Backend Data Synchronization Issue:
  - 1) Caching: The product detail page is cached (by a CDN or browser) and showing stale inventory data.
  - 2) WebSocket/Real-time Failure: The frontend is not receiving real-time inventory update pushes.
  - 3) API Lag: The inventory value was correct at the time the detail page API was called, but it changed before the order submission API was called.
3. SKU vs. Master Inventory Inconsistency: The product detail page might be displaying the total inventory for all SKUs (e.g., all colors of earphones = 1), but the user's selected SKU (e.g., "Black") has 0 inventory. The order submission API correctly checks SKU-level stock.
4. Inventory Locking Logic Bug: The cart might contain an item that was already

"locked" by a previous, unsubmitted order session for the same user, and the submission logic incorrectly includes that locked quantity.

**Investigation Plan:**

1. Reproduce the issue: Try to place an order for the same product immediately. Check if the inventory is truly 0 now.
2. Check Order History: Look for another order for the same product placed within the same minute as the user's attempt.
3. Compare Timestamps: Ask engineering to fetch logs for the user's GET /product/123 (detail page) request and their POST /order (submit) request. Calculate the time difference. If > a few seconds, concurrency or staleness is likely.
4. Check Product Type: Verify if the product has SKUs. If yes, confirm which specific SKU the user had in their cart and what the SKU-level inventory was at the time.
5. Check Caching Policy: Ask engineering what the cache TTL (Time to Live) is for the product detail API response. If it's long (e.g., 10 minutes), this is a problem.
6. Review Backend Logic: Confirm that the order submission API re-checks inventory against the primary database (the source of truth) and does not rely on any cached value. Confirm that inventory is locked at order creation, not at payment.
7. Check for Overselling Bug: Ask if the system uses optimistic or pessimistic locking for inventory. A bug here could allow two PENDING\_PAYMENT orders to be created for the last item.

## Part 3: Data and Metrics

### 9. Data Metrics Design

| Metric                         | Formula / Definition                                                                                     | Why It Matters                                                                                                             |
|--------------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Daily Active Users (DAU)       | Number of unique logged-in users who perform a key action (e.g., view product, add to cart).             | Measures the overall health and stickiness of the platform. Declining DAU is an early warning sign.                        |
| Product Detail Page Views (PV) | Total number of times product detail pages are loaded.                                                   | Measures top-of-funnel traffic and the effectiveness of product discovery (search, categories, marketing).                 |
| Add-to-Cart Rate (ATC)         | (Number of users who add at least one item to cart) / (Number of users who view a product detail page)   | Measures product attractiveness and pricing appeal. A low rate suggests issues with product content, price, or trust.      |
| Checkout Conversion Rate       | (Number of users who successfully submit an order) / (Number of users who viewed the cart/checkout page) | A critical metric for identifying friction in the checkout flow (e.g., complex forms, unexpected costs, slow performance). |

|                                |                                                                                                           |                                                                                                                                                       |
|--------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Payment Success Rate           | $\frac{\text{(Number of paid orders)}}{\text{(Number of submitted orders with PENDING\_PAYMENT status)}}$ | Measures the reliability of the payment gateway and the user's ability to pay. Low rates point to gateway integration issues or confusing payment UI. |
| Gross Merchandise Value (GMV)  | Total monetary value of all successfully PAID orders over a period (e.g., daily, weekly).                 | The primary measure of business sales scale and revenue potential.                                                                                    |
| Inventory Accuracy Rate        | $\frac{\text{(Number of orders without post-payment stockouts)}}{\text{(Total number of orders)}}$        | Measures the system's ability to track inventory correctly. A low rate causes cancellations, refunds, and poor customer experience.                   |
| Average Order Fulfillment Time | Average time taken from PAID status to SHIPPED status.                                                    | Measures operational efficiency. Long times indicate bottlenecks in the warehouse or logistics process, leading to customer dissatisfaction.          |